

PROPUESTA FORMAL DE PROYECTO FINAL

Asignatura: Programación Orientada a Objetos

LigaMX Tactical (Sistema de Gestión y Simulación Táctica)

Entorno de Desarrollo: Java

Profesor:

Ruben Gómez Olvera

Integrantes:

- Santiago Díaz Sánchez
- Christopher Alexander Valenzuela Cruz

ÍNDICE:

- 1.- Resumen ejecutivo
- 2.- Descripción del problema y justificación
- 3.- Usuario objetivo y alcance
- 4.- Objetivo del proyecto
 - 4.1 Objetivo General
 - 4.2 Objetivos Específicos
- 5.- Requerimientos y funcionalidades clave
- 6.- Arquitectura de software y diseño de clases
 - 6.1 Catálogo y Diagrama de Clases
 - 6.2 Aplicación de Conceptos POO (Herencia, Encapsulamiento, Polimorfismo)
- 7.- Manejo y control de excepciones
- 8.- Plan de prueba y casos de prueba
- 9.- Cronograma de implementación y entregables

1. RESUMEN EJECUTIVO

El proyecto **LigaMX Tactical** consiste en el diseño e implementación de un sistema gestor de plantillas futbolísticas y simulador táctico desarrollado en el lenguaje Java. El objetivo principal es proporcionar una herramienta que administre dinámicamente el rendimiento y estado físico de los futbolistas de un club, aplicando modelos matemáticos diferenciados según su posición en la cancha. La aplicación opera por interfaz de consola en el entorno IntelliJ y demuestra la aplicación sólida de los pilares de la Programación Orientada a Objetos: Abstracción, Encapsulamiento, Herencia y Polimorfismo.

2. DESCRIPCIÓN DEL PROBLEMA Y JUSTIFICACIÓN

2.1 El Problema

En el fútbol profesional moderno, la toma de decisiones tácticas se basa en métricas de rendimiento. Sin embargo, en torneos locales y ligas regionales, los cuerpos técnicos suelen utilizar herramientas estáticas (hojas de cálculo simples o anotaciones manuales). Esto presenta dos limitaciones principales:

1. **Falta de especificidad posicional:** No se evalúa el rendimiento con parámetros adecuados según la labor táctica (un portero requiere métricas distintas a las de un delantero).
2. **Ausencia de cálculo de fatiga:** No existe un seguimiento automatizado sobre cómo el desgaste físico paulatino en un encuentro decremente las probabilidades de éxito en las jugadas.

2.2 Justificación

Se requiere desarrollar un software dinámico que aplique estructuras de datos en memoria (ArrayList) para gestionar el plantel, evalúe el desempeño específico mediante clases especializadas e implemente un motor de simulación capaz de restar la resistencia de los atletas tras cada jugada en tiempo real.

3. USUARIO OBJETIVO Y ALCANCE

- **Usuario Objetivo:** directores técnicos, preparadores físicos, analistas de rendimiento y estudiantes del área de sistemas interesados en simulación deportiva.
- **Alcance del Sistema:**
 - Registro, consulta, modificación y baja de futbolistas.
 - Simulación por consola de eventos clave durante un partido.
 - Control de validaciones contra ingresos incoherentes de datos mediante excepciones.
 - *Límite:* El sistema se enfocará en la capa de lógica de negocios y consola sin incluir interfaz gráfica compleja (GUI) en esta etapa.

4. OBJETIVOS DEL PROYECTO

4.1 Objetivo General

Desarrollar una aplicación orientada a objetos en Java que administre la plantilla de un equipo de la Liga MX, valide los datos mediante excepciones personalizadas y simule encuentros evaluando jugadas de manera polimórfica y calculando la fatiga acumulada.

4.2 Objetivos Específicos

1. Estructurar una jerarquía de clases con una clase abstracta madre (Jugador) y clases especializadas (Portero, Defensa, Delantero).
2. Implementar colecciones dinámicas mediante `ArrayList` dentro de la clase administradora `ClubLigaMX`.
3. Sobrescribir el método abstracto `resolverJugadaClave()` para ejecutar polimorfismo dinámico durante el partido.
4. Crear la excepción custom `DatoDeportivoInvalidException` para el filtrado de errores de entrada.

5. REQUERIMIENTOS Y FUNCIONALIDADES CLAVE

1. **RF-01 | Alta de Futbolistas:** Captura de datos personales y atributos específicos según posición.
2. **RF-02 | Búsqueda por Dorsal:** Localización directa de un jugador en la lista dinámicamente.
3. **RF-03 | Modificación de Atributos:** Ajuste de parámetros de resistencia física y nivel de forma.
4. **RF-04 | Eliminación de Plantilla:** Desactivación o baja de un futbolista por su dorsal.
5. **RF-05 | Motor de Simulación Táctica:** Generación de eventos de partido que invocan métodos polimórficos.
6. **RF-06 | Módulo de Fatiga:** Disminución progresiva de la resistencia tras cada acción procesada.
7. **RF-07 | Gestión de Excepciones:** Control de datos numéricos fuera de rango y duplicidad de camisetas.
8. **RF-08 | Emisión de Reportes:** Impresión del resultado final y estado físico posterior al encuentro.

6. ARQUITECTURA DE SOFTWARE Y DISEÑO DE CLASES

6.1 Catálogo de Clases

Nombre de la Clase	Tipo / Estructura	Responsabilidad Principal	Atributos Representativos
Jugador	Abstracta (Padre)	Modelo base para todos los integrantes de la plantilla.	id, nombre, dorsal, resistencia, nivelForma
Portero	Concreta (Hija)	Lógica especializada en la atajada de balones y penales.	reflejos, efectividadPenales
Defensa	Concreta (Hija)	Lógica de marcaje, recuperación e intercepción.	fuerzaEntrada, intercepciones
Delantero	Concreta (Hija)	Lógica de definición y tiro a portería.	definicion, potenciaDisparo
ClubLigaMX	Gestora	Administra la colección <code>ArrayList<Jugador></code> del plantel.	nombreClub, plantilla, historialPartidos
Partido	Controladora	Procesa la simulación, el marcador y el desgaste.	folio, marcadorLocal, marcadorVisita
DatoDeportivoInvalidException	Custom Exception	Captura errores de negocio (rangos o valores incoherentes).	mensajeError
MenuConsola	Interfaz (Consola)	Despliega el menú interactivo para la lectura con Scanner.	scanner, club
Main	Clase Principal	Arranca la ejecución de la aplicación.	args

6.2 Aplicación de Conceptos de POO

- **Encapsulamiento:** Todos los atributos fueron declarados con el modificador `private`. El acceso y modificación de los datos se realiza estrictamente a través de métodos de lectura (getters) y escritura (setters) previamente validados.
- **Herencia:** Las clases `Portero`, `Defensa` y `Delantero` extienden de `Jugador`, heredando todos sus atributos generales y reduciendo la duplicidad de código.
- **Polimorfismo:** En la clase madre `Jugador` se declara: `public abstract int resolverJugadaClave(int intensidadRival);` Cada clase hija sobrescribe este método con su propia fórmula. Durante la simulación en `Partido`, se recorre la lista con un bucle `for` llamando a `jugador.resolverJugadaClave()`. La Máquina Virtual de Java (JVM) ejecuta en tiempo de ejecución la versión exacta según el objeto real almacenado, sin requerir múltiples condicionales `if/else`.

7. MANEJO Y CONTROL DE EXCEPCIONES

1. **`DatoDeportivoInvalidException`:** Se lanza cuando el usuario ingresa un valor de resistencia menor a 0 o mayor a 100, o cuando los valores de forma no corresponden a la escala válida.
2. **`IllegalArgumentException`:** Se dispara al intentar registrar dos futbolistas con el mismo número de dorsal dentro del mismo `ClubLigaMX`.
3. **`NullPointerException`:** Se controla cuando se intenta buscar o modificar a un jugador con un dorsal que no existe dentro de la lista del equipo.

8. PLAN DE PRUEBAS (CASOS DE PRUEBA)

- **Caso de Prueba 1: Filtrado de Datos Erróneos**
 - *Acción:* Intentar dar de alta a un jugador con -25 en el campo de resistencia.
 - *Resultado Esperado:* El sistema intercepta el error con `DatoDeportivoInvalidException`, muestra la advertencia en consola y regresa al menú sin romper el programa.
- **Caso de Prueba 2: Control de Dorsales Únicos**
 - *Acción:* Registrar a un `Portero` con el dorsal 1 y posteriormente a un `Defensa` con el mismo dorsal 1.
 - *Resultado Esperado:* El sistema detecta la duplicidad y bloquea la inserción en el `ArrayList`.
- **Caso de Prueba 3: Comprobación de Polimorfismo**
 - *Acción:* Ejecutar un encuentro simulado teniendo registrados un `Portero`, un `Defensa` y un `Delantero`.
 - *Resultado Esperado:* Cada evento en consola calcula la jugada utilizando los datos únicos (`reflejos`, `fuerzaEntrada`, `definicion`) según corresponda a la subclase activa.
- **Caso de Prueba 4: Verificación del Módulo de Fatiga**
 - *Acción:* Consultar el estado de la plantilla antes y después de simular el partido.

- *Resultado Esperado:* Comprobar que la variable resistencia del objeto ha disminuido respecto a su valor inicial.

9. CRONOGRAMA DE IMPLEMENTACIÓN Y ENTREGABLES

- **Semana 1:** Implementación de la jerarquía de clases base (Jugador, Portero, Defensa, Delantero) en IntelliJ. Creación de la estructura del ArrayList en ClubLigaMX.
- **Semana 2:** Programación de la clase Partido, codificación del método polimórfico resolverJugadaClave() y desarrollo de la lógica de desgaste/fatiga.
- **Semana 3:** Integración de la clase DatoDeportivoInvalidException, refinamiento del menú por consola en MenuConsola y ejecución de la batería de pruebas unitarias.